МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ

по лабораторной работе №2

по дисциплине «Вычислительная математика»

Тема: Метод Гаусса

Студент гр. 4343 _____ Едина А.С.

Преподаватель _____ Пуеров Г. Ю.

Санкт-Петербург

2026

Задание

В ходе выполнения работы студенты должны найти решение системы линейных уравнений с n неизвестными, заданной матрицей коэффициентов и вектором свободных членов, методом Гаусса.

Теоретическая часть

Рассматривается система линейных уравнений n-го порядка

$$\begin{gathered} a_{11}x_1 + a_{12}x_2 + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{2n}x_n = b_2 \\ . \quad . \quad . \quad . \quad . \\ a_{n1}x_1 + a_{n2}x_2 + a_{\mathrm{nn}}x_n = b_n, \end{gathered} \tag{2.1}$$

что в векторном виде записывается как $\mathrm{A}\mathrm{x} = b$.

Суть метода исключения по главным элементам (метод Гаусса) заключается в следующем. Находится наибольший по абсолютной величине коэффициент a_{kj}. Для исключения x_j из i-го уравнения $(i \neq k)$ необходимо умножить k-е уравнение на $a_{\mathrm{ij}}/a_{\mathrm{kj}}$ и вычесть его из i-го уравнения, после чего процесс повторяется для исключения другого неизвестного из оставшихся n-1 уравнений и т. д. В результате система (2.1) приводится к треугольному виду

$$\begin{gathered} x_1 + \alpha_{12}x_2 + \ldots + \alpha_{1n}x_n = \beta_n \\ x_2 + \ldots + \alpha_{2n}x_n = \beta_n \\ . \quad . \quad . \quad . \quad . \quad . \\ x_n = \beta_n, \end{gathered} \tag{2.2}$$

из которого легко находятся неизвестные $x_1,\ldots,x_n$. Процесс приведения системы к виду (2.2) называется прямым ходом, а нахождение неизвестных $x_1,\ldots,x_n$ - обратным ходом метода Гаусса.

Следует отметить, что если матрица заданной системы вырожденная, то перед исключением некоторой неизвестной главный элемент a_{kj} окажется равным нулю, что и будет свидетельствовать о равенстве нулю определителя системы.

Тестирование программы

Таблица 1 – Результаты тестирования

№ п/п	Входные данные	Выходные данные
1.	./generate 10 1 test_10.txt сгенерирована случайная матрица с диагональным преобладанием данные записаны в test_10.txt root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./gauss test_10.txt 10 -219.9831738229 37.2887934519 - 46.9023282870 - 3.1026988708 2.6831608819 - 17.2222025661 - 13.9405379844 13.2537606556 20.1548561690 - 27.0251246789 - 44.8027045268 -3.6830907315 180.7452032417 5.7699176500 - 8.3124147266 - 40.5076082572 - 7.3613383255 - 4.5594298863 26.2090250967 46.7423241569 - 24.2916577489 5.7877898470 -40.9863557681 - 25.1748930485 - 239.0728862036 - 46.5285215103 - 2.0207690857 37.2626268928 - 13.2715714000 - 24.8207703751 20.1283924095 - 25.5339243595 23.9297529015 3.3770824767 42.0080519808 24.3358605638 -	размер матрицы: 10x10 решаем систему система успешно решена за 6.155e-06 сек. РЕЗУЛЬТАТЫ норма невязки L2: 1.865175e-14 норма невязки L_inf: 1.421085e-14 вектор решений x: x[0] = 0.208373 x[1] = -0.004619 x[2] = -0.124163 x[3] = -0.115853 x[4] = -0.138830 x[5] = 0.040496 x[6] = 0.138644 x[7] = 0.025520 x[8] = 0.134772 x[9] = 0.186003 размер матрицы: 10x10 решаем систему методом Eigen Eigen решил систему за 5.9763e-05 сек. результаты EIGEN норма невязки L2: 1.432145e-14 норма невязки L_inf: 7.105427e-15 Сравнение времени: Метод Гаусса: 59763 нс Метод Eigen: 61555 нс

	314.8483955553 - 12.5366756709 - 19.5403970972 42.9211029397 - 41.3766561869 - 46.6682140772 35.8017877714 40.1777588944 -25.9157912836 15.3802040823 39.7193501658 36.3319082751 - 277.8546829938 - 10.3888094168 42.6595971822 4.1930534046 26.1082369380 45.0380686873 41.4593333114 4.0340874404 36.0001099568 41.2295721592 16.5233570270 16.5798409460 259.0048209357 30.4816174340 44.5630477642 28.0883841800 12.5636803138 13.3134542019 -37.2777459690 - 14.5786389220 43.3346871258 23.2040715549 - 2.1938386185 - 27.6836598701 - 225.6647473217 - 8.6776646506 44.8101908739 - 13.8719424831 - 44.6351583401 38.8533934447 48.4277547305 - 44.0202721569 18.3413132786 - 39.4468205487 - 3.0142516250 - 7.3086459835 - 279.9918658841 4.8967691640 49.5275588170 18.2811076456	

	-22.6178603857 11.1741886718 14.9281829798 20.0473272498 42.2564688916 - 15.2869841682 22.9712449354 49.0831476539 - 269.7471302221 49.9794467996 - 38.0468791365 17.6187661499 20.1077640084 21.4313651103 - 46.4681823927 - 37.6865063301 - 12.7340333855 9.5706263623 5.8455134384 41.0371167734 - 249.3271383195 - 28.3516330632	
2.	Матрица размером 5000x5000 ./generate 5000 1 test_5000.txt сгенерирована случайная матрица с диагональным преобладанием данные записаны в test_5000.txt	./gauss test_5000.txt размер матрицы: 5000x5000 решаем систему система успешно решена за 28.0832 сек. РЕЗУЛЬТАТЫ норма невязки L2: 7.611070e-12 норма невязки L_inf: 4.973799e-13 (матрица слишком большая для вывода вектора решений) ./eigen_solver test_5000.txt размер матрицы: 5000x5000 решаем систему методом Eigen Eigen решил систему за 4.73188 сек. результаты EIGEN норма невязки L2: 3.477094e-12 норма невязки L_inf: 2.842171e-13 Сравнение времени: Метод Гаусса: 28 сек Метод Eigen: 5 сек
3.	Матрица Гильберта ./generate 10 2 test_g_10.txt сгенерирована матрица Гильберта данные записаны в test_g_10.txt	root@oumuamua:/mnt/c/uni/compmath/comp-math/lab2# ./gauss test_g_10.txt размер матрицы: 10x10 решаем систему система успешно решена за 1.584e-06 сек. РЕЗУЛЬТАТЫ

	10 1.0000000000 0.5000000000 0.3333333333 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 2.9289682540 0.5000000000 0.3333333333 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 2.0198773449 0.3333333333 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 1.6032106782 0.2500000000 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 1.3468004218 0.2000000000 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909	норма невязки L2: 3.845925e-16 норма невязки L_inf: 2.220446e-16 вектор решений x: x[0] = 0.999941 x[1] = 1.003910 x[2] = 0.937279 x[3] = 1.415873 x[4] = -0.364518 x[5] = 3.297939 x[6] = -0.709454 x[7] = 0.912663 x[8] = 1.850487 x[9] = 0.655828 ./eigen_solver test_g_10.txt размер матрицы: 10x10 решаем систему методом Eigen Eigen решил систему за 3.794e-06 сек. результаты EIGEN норма невязки L2: 7.447602e-16 норма невязки L_inf: 4.440892e-16 Сравнение времени: Метод Гаусса: 15844 нс Метод Eigen: 37944 нс

	0.0833333333 0.0769230769 0.0714285714 1.1682289932 0.1666666667 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 1.0348956599 0.1428571429 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 0.0625000000 0.9307289932 0.1250000000 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 0.0625000000 0.0588235294 0.8466953798 0.1111111111 0.1000000000 0.0909090909 0.0833333333 0.0769230769 0.0714285714 0.0666666667 0.0625000000 0.0588235294 0.0555555556 0.7772509353 0.1000000000 0.0909090909 0.0833333333 0.0769230769	

	0.0714285714 0.0666666667 0.0625000000 0.0588235294 0.0555555556 0.0526315789 0.7187714032	

Вывод

В результате выполнения работы реализован алгоритм Гаусса с частичным выбором ведущего элемента для решения СЛАУ. С помощью библиотеки Eigen проведён анализ обусловленности матриц. Предложенная реализация обеспечивает корректное решение систем для матриц различных размеров.

Приложения А

Исходный код gauss.cpp

```
#include <iostream>
#include <fstream>
#include <vector>
#include <cmath>
#include <chrono>
#include <iomanip>

using namespace std;

const int NMAX = 5000;

int GAUSS(const vector<vector<double>>& a_in, const vector<double>& b_in,
vector<double>& x, int n) {
    if (n < 2) return 2;
    if (n > NMAX) return 3;

    vector<vector<double>> a = a_in;
    vector<double> b = b_in;
    x.assign(n, 0.0);

    for (int i = 0; i < n; ++i) {

        int pivot_row = i;
        double max_val = std::abs(a[i][i]);

        for (int k = i + 1; k < n; ++k) {
            if (std::abs(a[k][i]) > max_val) {
                max_val = std::abs(a[k][i]);
                pivot_row = k;
            }
        }

        if (max_val < 1e-12) {
            return 1;
        }

        if (pivot_row != i) {
            swap(a[i], a[pivot_row]);
            swap(b[i], b[pivot_row]);
        }

        for (int k = i + 1; k < n; ++k) {
            double factor = a[k][i] / a[i][i];
            for (int j = i; j < n; ++j) {
                a[k][j] -= factor * a[i][j];
            }
            b[k] -= factor * b[i];
        }
    }

    for (int i = n - 1; i >= 0; --i) {
        double sum = 0.0;
        for (int j = i + 1; j < n; ++j) {
            sum += a[i][j] * x[j];
        }
        x[i] = (b[i] - sum) / a[i][i];
```

```
    }

    return 0;
}

int main(int argc, char* argv[]) {
    if (argc != 2) {
        cout << "использование: " << argv[0] << " <файл_с_матрицей>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if (!in.is_open()) {
        cerr << "ошибка: не удалось открыть файл.\n";
        return 1;
    }

    int n;
    in >> n;

    vector<vector<double>> A(n, vector<double>(n));
    vector<double> b(n);
    vector<double> x;

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            in >> A[i][j];
        }
        in >> b[i];
    }
    in.close();

    cout << "размер матрицы: " << n << "x" << n << "\n";
    cout << "решаем систему\n";

    auto start_time = chrono::high_resolution_clock::now();

    int status = GAUSS(A, b, x, n);

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end_time - start_time;

    if (status == 1) {
        cout << "ошибка: Матрица вырождена!\n";
    } else if (status == 2) {
        cout << "ошибка: n < 2\n";
    } else if (status == 3) {
        cout << "ошибка: n > NMAX (" << NMAX << ")\n";
    } else if (status == 0) {
         cout << "система успешно решена за " << elapsed.count() << "
сек.\n";

        double norm_L2 = 0.0;
        double norm_Linf = 0.0;

        for (int i = 0; i < n; ++i) {
            double ax_val = 0.0;
            for (int j = 0; j < n; ++j) {
```

```
                ax_val += A[i][j] * x[j];
            }
            double residual = std::abs(ax_val - b[i]);

            norm_L2 += residual * residual;
            if (residual > norm_Linf) norm_Linf = residual;
        }
        norm_L2 = std::sqrt(norm_L2);

        cout << "\nРЕЗУЛЬТАТЫ\n";
        cout << scientific << setprecision(6);
        cout << "норма невязки L2: " << norm_L2 << "\n";
        cout << "норма невязки L_inf: " << norm_Linf << "\n";

        if (n <= 20) {
            cout << "\nвектор решений x:\n";
            for (int i = 0; i < n; ++i) {
                cout << "x[" << i << "] = " << fixed << setprecision(6)
<< x[i] << "\n";
            }
        } else {
              cout << "\n(матрица слишком большая для вывода вектора
решений)\n";
        }
    }

    return 0;
}
```

Исходный код eigen_solver.cpp:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <iomanip>
#include <eigen3/Eigen/Dense>

using namespace std;
using namespace Eigen;

int main(int argc, char* argv[]) {
    if (argc != 2) {
        cout << "Использование: " << argv[0] << " <файл_с_матрицей>\n";
        return 1;
    }

    ifstream in(argv[1]);
    if (!in.is_open()) {
        cerr << "ошибка: не удалось открыть файл.\n";
        return 1;
    }

    int n;
    in >> n;

    MatrixXd A(n, n);
    VectorXd b(n);

    for (int i = 0; i < n; ++i) {
```

```
        for (int j = 0; j < n; ++j) {
            in >> A(i, j);
        }
        in >> b(i);
    }
    in.close();

    cout << "размер матрицы: " << n << "x" << n << "\n";
    cout << "решаем систему методом Eigen\n";

    auto start_time = chrono::high_resolution_clock::now();

    VectorXd x = A.partialPivLu().solve(b);

    auto end_time = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end_time - start_time;

    cout << "Eigen решил систему за " << elapsed.count() << " сек.\n";

    VectorXd residual = A * x - b;
    double norm_L2 = residual.norm();
    double norm_Linf = residual.lpNorm<Infinity>();

    cout << "\nрезультаты EIGEN\n";
    cout << scientific << setprecision(6);
    cout << "норма невязки L2: " << norm_L2 << "\n";
    cout << "норма невязки L_inf: " << norm_Linf << "\n";

    return 0;
}
```

Исходный код generate.cpp:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <random>
#include <cmath>
#include <iomanip>
#include <string>

using namespace std;

void generateRandomDiagonallyDominant(int n, vector<vector<double>>& A,
vector<double>& b) {
    random_device rd;
    mt19937 gen(rd());
    uniform_real_distribution<double> dist(-50.0, 50.0);

    for (int i = 0; i < n; ++i) {
        double sum = 0.0;
        for (int j = 0; j < n; ++j) {
            if (i != j) {
                A[i][j] = dist(gen);
                sum += std::abs(A[i][j]);
            }
        }
        A[i][i] = sum + std::abs(dist(gen)) + 1.0;
        if (dist(gen) < 0) A[i][i] = -A[i][i];
```

```
        b[i] = dist(gen);
    }
}

void generateHilbert(int n, vector<vector<double>>& A, vector<double>&
b) {
    for (int i = 0; i < n; ++i) {
        b[i] = 0.0;
        for (int j = 0; j < n; ++j) {
            A[i][j] = 1.0 / (i + j + 1.0);
            b[i] += A[i][j];
        }
    }
}

int main(int argc, char* argv[]) {
    if (argc != 4) {
        cout << "неверные арги\n";
        return 1;
    }

    int n = stoi(argv[1]);
    int type = stoi(argv[2]);
    string filename = argv[3];

    vector<vector<double>> A(n, vector<double>(n));
    vector<double> b(n);

    if (type == 1) {
        generateRandomDiagonallyDominant(n, A, b);
           cout << "сгенерирована случайная матрица с диагональным
преобладанием\n";
    } else if (type == 2) {
        generateHilbert(n, A, b);
        cout << "сгенерирована матрица Гильберта\n";
    } else {
        cout << "введенно неверное значение\n";
        return 1;
    }

    ofstream out(filename);
    if (!out.is_open()) {
        cerr << "ошибка создания файла!\n";
        return 1;
    }

    out << n << "\n";
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            out << fixed << setprecision(10) << A[i][j] << " ";
        }
        out << b[i] << "\n";
    }

    out.close();
    cout << "данные записаны в " << filename << "\n";
    return 0;
```

}